
React UI Setup & Styling with HTML/Centric CSS

Welcome to the updated practical lab for Hour 2! Now that you have refreshed your knowledge of **HTML structures** and **CSS styling**, you will apply those principles directly inside your React components by creating a styled prediction card layout.

1. Concept Overview

In this lab, you will:

- Structure your React components using semantic HTML tags (`<header>` , `<main>` , `<section>` , `<form>`).
 - Apply CSS styling using external stylesheets (`.css`) to control layout, colors, and the CSS Box Model.
 - Use **Vite** to preview your changes in real-time with Hot Module Replacement (HMR).
-

2. Step-by-Step Lab Instructions

1. **Download and Install Node.js:** Required foundation. Go to the official [Node.js website](https://nodejs.org/) and download the **LTS (Long Term Support)** installer for your operating system (Windows, Mac, or Linux). Run the downloaded installer and accept the default settings. This installs both Node.js and NPM on your machine.
2. **Verify the Installation:** Crucial check. Open your computer's terminal (Command Prompt/PowerShell on Windows, Terminal on Mac). Type `node -v` and press Enter. Then type `npm -v` and press Enter. If both commands return version numbers (e.g., `v20.x.x`), your installation was successful.
3. **Scaffold the React Project:** `npm create vite@latest fullstack-ml-ui -- --template react`. In your terminal, navigate to the folder where you want your project to live (using the `cd` command, like `cd Documents`). Then, run the Vite creation command. Replace `fullstack-ml-ui` with whatever you want to name your project folder.

4. **Navigate into the Project:** `cd fullstack-ml-ui`. Move your terminal's active directory into the newly created project folder so the next commands apply to it.
5. **Download the Dependencies:** `npm install`. Run this command to tell NPM to read your project's blueprint and download all the necessary React and Vite code from the internet into a local `node_modules` folder. This might take a minute depending on your connection.
6. **Start the Development Server:** `npm run dev`. Launch the local server. Your terminal will display a local network address (usually `http://localhost:5173`).

Once the server is running, hold `Ctrl` (or `Cmd` on Mac) and click the `http://localhost:5173` link in your terminal, or copy and paste it into your web browser. You will see the default Vite + React starter page with a clickable counter button.

Step 1: Review Project Files

Open your `fullstack-ml-ui` project in **Visual Studio Code**. Ensure your development server is running:

```
npm run dev
```

Step 2: Create a Styled Header Component

1. Create `src/components` folder, Inside `src/components/Header.jsx`, update the code to use semantic HTML and a dedicated class name:

```
// src/components/Header.jsx
import './Header.css';

function Header() {
  return (
    <header className="app-header">
      <h1>AI-Integrated Full-Stack Dashboard</h1>
      <p>React Frontend Connected to FastAPI & Machine Learning</p>
    </header>
  );
}

export default Header;
```

2. Create a styling file named `src/components/Header.css` and apply CSS Flexbox and Box Model properties:

```
/* src/components/Header.css */
.app-header {
  background-color: #0f172a;
  color: #ffffff;
  padding: 1.5rem 2rem;
```

```

    text-align: center;
    border-bottom: 4px solid #3b82f6;
}

.app-header h1 {
    margin: 0 0 0.5rem 0;
    font-size: 1.75rem;
}

.app-header p {
    margin: 0;
    color: #94a3b8;
    font-size: 0.95rem;
}

```

Step 3: Build a Styled ML Input Form Component

Next, let's build a semantic HTML form styled with CSS to capture user input features for an upcoming machine learning model (e.g., house price or student score prediction).

1. Create a new file named `src/components/PredictionForm.jsx` :

```

// src/components/PredictionForm.jsx
import './PredictionForm.css';

function PredictionForm() {
    return (
        <main className="form-container">
            <section className="card">
                <h2>Model Feature Input</h2>
                <form onSubmit={(e) => e.preventDefault()}>
                    <div className="input-group">
                        <label htmlFor="feature1">Primary Feature (e.g., Sq Ft /
Hours):</label>
                        <input type="number" id="feature1" placeholder="Enter
value..." />
                    </div>

                    <div className="input-group">
                        <label htmlFor="feature2">Secondary Feature (e.g., Rooms /
Age):</label>
                        <input type="number" id="feature2" placeholder="Enter
value..." />
                    </div>

                    <button type="submit" className="submit-btn">Run
Prediction</button>
                </form>
            </section>
        </main>
    );
}

```

```
}
```

```
export default PredictionForm;
```

2. Create a companion stylesheet named `src/components/PredictionForm.css`:

```
/* src/components/PredictionForm.css */
```

```
.form-container {  
  display: flex;  
  justify-content: center;  
  padding: 3rem 1rem;  
  background-color: #f8fafc;  
  min-height: calc(100vh - 120px);  
}  
  
.card {  
  background-color: #ffffff;  
  padding: 2rem;  
  border-radius: 12px;  
  box-shadow: 0 4px 6px -1px rgba(0, 0, 0, 0.1), 0 2px 4px -1px rgba(0, 0,  
0, 0.06);  
  width: 100%;  
  max-width: 450px;  
}  
  
.card h2 {  
  margin-top: 0;  
  color: #1e293b;  
  font-size: 1.25rem;  
  margin-bottom: 1.5rem;  
}  
  
.input-group {  
  margin-bottom: 1.25rem;  
  display: flex;  
  flex-direction: column;  
}  
  
.input-group label {  
  font-size: 0.875rem;  
  font-weight: 600;  
  color: #475569;  
  margin-bottom: 0.5rem;  
}  
  
.input-group input {  
  padding: 0.75rem;  
  border: 1px solid #cbd5e1;  
  border-radius: 6px;  
  font-size: 1rem;  
  outline: none;  
  transition: border-color 0.2s;
```

```

}

.input-group input:focus {
  border-color: #3b82f6;
}

.submit-btn {
  width: 100%;
  background-color: #3b82f6;
  color: white;
  border: none;
  padding: 0.75rem;
  font-size: 1rem;
  font-weight: 600;
  border-radius: 6px;
  cursor: pointer;
  transition: background-color 0.2s;
}

.submit-btn:hover {
  background-color: #2563eb;
}

```

Step 4: Integrate Components into App.jsx

Open `src/App.jsx` and assemble your layout:

```

// src/App.jsx
import Header from "../components/Header";
import PredictionForm from "../components/PredictionForm";

function App() {
  return (
    <div>
      <Header />
      <PredictionForm />
    </div>
  );
}

export default App;

```

Save all files and inspect your browser window. You now have a clean, structured, and professionally styled UI layout ready to capture user inputs for your machine learning model!

3. Reference Links & Further Reading

- [MDN CSS Flexbox Guide](#): Learn more about aligning layout elements efficiently.

- **React Styling Documentation:** Best practices for organizing CSS styles in modern React apps.
-